
PART 1: PATTERN MATCHING IN PHP (REGEX)

✅ What is Pattern Matching?

Pattern matching means:

👉 **Searching or validating text using a specific pattern**

PHP uses **Regular Expressions (Regex)** for this.

◆ Why we use Pattern Matching?

- Validate email ✓
 - Validate phone number ✓
 - Check password strength ✓
 - Extract data from strings ✓
-

◆ Basic Syntax

`preg_match(pattern, string)`

- Returns **1** → match found
 - Returns **0** → no match
-

◆ Understanding REGEX (VERY IMPORTANT)

Symbol Meaning

`/.../` Pattern delimiter

`^` Start of string

`$` End of string

`.` Any character

`*` 0 or more

`+` 1 or more

Symbol Meaning

?	optional
[a-z]	lowercase letters
[0-9]	digits
\.	actual dot
\d	digit
\s	whitespace

Example 1: Simple Match

```
<?php
$text = "Hello Bro";

if (preg_match("/Bro/", $text)) {
    echo "Match Found";
} else {
    echo "No Match";
}

?>
```

👉 Output: Match Found

Example 2: Validate Email

```
<?php
$email = "student123@gmail.com";

$pattern = "/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}$/";
if (preg_match($pattern, $email)) {
    echo "Valid Email";
} else {
```

```
    echo "Invalid Email";
}
?>
```

Explanation

Part	Meaning
^	start
[a-zA-Z0-9._%+-]+	username
@	must have @
[a-zA-Z0-9.-]+	domain
\.	dot
[a-z]{2,}	extension
\$	end

Example 3: Phone Number Validation

```
<?php
$phone = "9876543210";

if (preg_match("/^[0-9]{10}$/", $phone)) {
    echo "Valid Phone Number";
} else {
    echo "Invalid Phone Number";
}
?>
```

 Must be exactly 10 digits

Example 4: Password Validation (Important)

```
<?php
$password = "Pass@123";

$pattern = "/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$$%^&*]).{8,}$/";

if (preg_match($pattern, $password)) {
    echo "Strong Password";
} else {
    echo "Weak Password";
}
?>
```

💡 Rules:

- At least 1 uppercase
 - At least 1 lowercase
 - At least 1 number
 - At least 1 special character
 - Minimum 8 characters
-

◆ Other Functions

1. Replace Pattern

```
preg_replace("/cat/", "dog", "I have a cat");
```

👉 Output: I have a dog

2. Split String

```
preg_split("/,/ ", "apple,banana,mango");
```

👉 Output: Array ['apple','banana','mango']

📄 PART 2: FORM VALIDATION IN PHP

✅ What is Form Validation?

👉 Checking whether user input is:

- Correct ✓
- Safe ✓
- Complete ✓

◆ Types of Validation

1. Client-side (JavaScript)

2. Server-side (PHP) ✅ (IMPORTANT)

FULL EXAMPLE (IMPORTANT FOR EXAM)

📄 HTML FORM

```
<form method="POST" action="">

  Name: <input type="text" name="name"><br><br>
  Email: <input type="text" name="email"><br><br>
  Phone: <input type="text" name="phone"><br><br>
  Password: <input type="password" name="password"><br><br>

  <input type="submit" name="submit" value="Register">

</form>
```

🧠 PHP VALIDATION CODE

```
<?php

if (isset($_POST['submit'])) {

    $name = $_POST['name'];
```

```

$email = $_POST['email'];

$phone = $_POST['phone'];

$password = $_POST['password'];


// Name Validation
if (empty($name)) {
    echo "Name is required <br>";
}

// Email Validation
if (!preg_match("/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}$/", $email)) {
    echo "Invalid Email <br>";
}

// Phone Validation
if (!preg_match("/^[0-9]{10}$/", $phone)) {
    echo "Invalid Phone Number <br>";
}

// Password Validation
if (!preg_match("/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&*]).{8,}$/", $password)) {
    echo "Weak Password <br>";
}

// If everything is correct
if (!empty($name) &&
    preg_match("/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}$/", $email) &&
    preg_match("/^[0-9]{10}$/", $phone) &&
    preg_match("/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&*]).{8,}$/", $password)
) {
    echo "Registration Successful ";
}
}

```

?>

Important Concepts

◆ 1. isset()

`isset($_POST['submit'])`

👉 Checks if form is submitted

◆ 2. empty()

`empty($name)`

👉 Checks if field is empty

◆ 3. \$_POST

👉 Used to get form data

💡 Common Validation Rules (Exam Point)

Field	Rule
Name	Only letters
Email	Proper format
Phone	10 digits
Password	Strong

🎯 Final Summary

Pattern Matching

- Uses `preg_match()`
- Helps validate data
- Uses regex

Form Validation

- Uses \$_POST
 - Uses regex for validation
 - Uses isset() and empty()
-

1. FILE HANDLING IN PHP

What is File Handling?

 File handling means:

Creating, reading, writing, and managing files using PHP

◆ Why we use File Handling?

- Store data permanently 
 - Log user activity 
 - Save form data 
-

◆ Basic Functions

Function	Purpose
fopen()	Open file
fwrite()	Write file
fread()	Read file
fclose()	Close file
file_exists()	Check file
unlink()	Delete file

◆ File Modes (VERY IMPORTANT)

Mode Meaning

"r" Read only

"w" Write (overwrite)

"a" Append

"x" Create new file

🔥 Example 1: Create & Write File

```
<?php
$file = fopen("data.txt", "w"); // create/open

fwrite($file, "Hello Bro!\n");
fwrite($file, "Learning PHP");
fclose($file);

?>
```

👉 If file exists → overwritten

👉 If not → created

🔥 Example 2: Read File

```
<?php
$file = fopen("data.txt", "r");

$content = fread($file, filesize("data.txt"));
echo $content;

fclose($file);

?>
```

Example 3: Append Data

```
<?php
$file = fopen("data.txt", "a");

fwrite($file, "\nNew Line Added");

fclose($file);
?>
```

Example 4: Check File Exists

```
<?php
if (file_exists("data.txt")) {
    echo "File exists";
} else {
    echo "File not found";
}
?>
```

Example 5: Delete File

```
<?php
unlink("data.txt");
?>
```

Important Points

- Always use fclose()
 - Handle file errors
 - Use correct mode
-

2. COOKIES IN PHP

✅ What is a Cookie?

👉 A cookie is:

Small data stored in the user's browser

♦ Why Cookies?

- Remember user login 
 - Store preferences 
 - Track users 
-

♦ Syntax

```
setcookie(name, value, expiry_time);
```

🔥 Example 1: Set Cookie

```
<?php  
setcookie("user", "Naveen", time() + 3600);  
echo "Cookie Set";  
?>
```

👉 Valid for 1 hour

🔥 Example 2: Access Cookie

```
<?php  
echo $_COOKIE['user'];  
?>
```

🔥 Example 3: Delete Cookie

```
<?php
```

```
setcookie("user", "", time() - 3600);  
?>
```

Example 4: Check Cookie

```
<?php  
if (isset($_COOKIE['user'])) {  
    echo "Welcome " . $_COOKIE['user'];  
} else {  
    echo "Cookie not set";  
}  
?>
```

Important Points

- Stored on browser
 - Less secure
 - Limited size (~4KB)
-

3. SESSION TRACKING IN PHP

What is a Session?

 Session stores data:

On the server (more secure than cookies)

◆ Why Sessions?

- Maintain login session 
 - Store user data securely
 - Track user activity
-

◆ Basic Functions

Function	Purpose
session_start()	Start session
\$_SESSION	Store data
session_destroy()	Delete session

Example 1: Start Session

```
<?php
session_start();
?>
```

 MUST be first line

Example 2: Store Data

```
<?php
session_start();

$_SESSION['username'] = "Naveen";
echo "Session created";
?>
```

Example 3: Access Session

```
<?php
session_start();

echo $_SESSION['username'];
?>
```

Example 4: Destroy Session

```
<?php
```

```
session_start();  
session_destroy();  
?>
```

Example 5: Remove Specific Variable

```
<?php  
session_start();  
unset($_SESSION['username']);  
?>
```

Real-Life Example (Login System)

login.php

```
<?php  
session_start();  
  
$_SESSION['user'] = "Admin";  
  
echo "Login Successful";  
?>
```

dashboard.php

```
<?php  
session_start();  
  
if (isset($_SESSION['user'])) {  
    echo "Welcome " . $_SESSION['user'];  
} else {  
    echo "Please login";  
}
```

```
}  
?>
```

Cookies vs Sessions (IMPORTANT)

Feature Cookies Sessions

Storage Browser Server

Security Less More

Size Limited Large

Lifetime Fixed time Until browser close

Important Notes (Exam Points)

- Cookies → `setcookie()`
 - Sessions → `session_start()`
 - Always start session before output
 - Use `isset()` to check
-

FINAL SUMMARY

File Handling

- Create, read, write files
- Functions: `fopen()`, `fwrite()`, `fread()`

Cookies

- Stored in browser
- Used for user preferences

Sessions

- Stored in server
 - Used for login systems
-